

Resumen de Arquitectura de Computadores

Clase #12 - Martes 7 de abril de 2026

Fuentes usadas: apuntes PDF 4-7.pdf y revisión del video abril-7.mp4. Los tiempos del video son aproximados.

Idea central de la clase

La clase gira alrededor de un programa MASM/x64 que verifica una identidad matemática: **la suma de los cubos de los primeros n enteros es igual al cuadrado de la suma de esos enteros**. El objetivo no es solo calcularla, sino practicar paso de parámetros, valores de retorno, stack frame, variables locales, ciclos, comparación y depuración en Visual Studio.

$$1^3 + 2^3 + \dots + n^3 = (1 + 2 + \dots + n)^2 = (n(n+1) / 2)^2$$

Mapa rápido del video

Tiempo aprox.	Contenido observado
00:00-10:00	Repaso de la identidad matemática y explicación de la fórmula.
10:00-17:00	Validación visual en Excel: se comparan suma de cubos y cuadrado de la suma para varios n.
20:00-35:00	Construcción del main en Visual Studio: llamadas a módulos, uso de R8/R9 y comparación final.
40:00-45:00	Dibujo del stack frame clásico y comparación entre pasar datos por pila y por registros.
55:00-80:00	Implementación de getSumLimit y sumCubes; creación de variables locales con sub RSP.
85:00-100:00	Depuración paso a paso: ventana de registros, memoria, RSP/RBP y valores de la pila.
100:00-105:00	Se observa formulaCuadradoSuma pendiente de implementación y se limpia la versión de sumCubes.

Estructura general del programa

El programa principal trabaja así:

- Pide o simula el límite `n` usando `getSumLimit`.
- Calcula la suma de cubos hasta `n` con `sumCubes` y deja el resultado en `R8d`.
- Calcula la fórmula del cuadrado de la suma con `formulaCuadradoSuma` y deja el resultado en `R9d`.
- Compara `R8` y `R9` con `cmp`.
- Si son distintos, salta al `else`; si son iguales, llama al procedimiento de igualdad.

```
main PROC
    sub RSP, 8
    call getSumLimit
    pop RCX
    mov n, CX

    xor R8, R8
    call sumCubes

    xor R9, R9
    call formulaCuadradoSuma

    cmp R8, R9
    jne elseComparar
    call anunciarIgualdad
    jmp endIfResultadosIguales
elseComparar:
    call anunciarDesigualdad
endIfResultadosIguales:
    call ExitProcess
main ENDP
```

Stack frame y paso de datos entre módulos

Punto clave: el caller crea el espacio para parámetros y valor retornado; el hardware agrega la dirección de retorno cuando se ejecuta `call`; y el called crea sus variables locales si las necesita.

- `RSP` apunta al tope actual de la pila.
- La pila de Intel crece hacia direcciones menores: reservar espacio se hace con `sub RSP, cantidad`.
- `call` empuja la dirección de retorno en la pila y luego salta al procedimiento.
- `ret` toma la dirección en `[RSP]` y vuelve a esa instrucción.
- Si el procedimiento creó variables locales, debe restaurar `RSP` antes de hacer `ret`.

En esta clase se contrastan tres ideas:

Caso	Qué queda en pila	Qué va en registros
Stack frame clásico	Variables locales, dirección de	No es la estrategia principal.

	retorno, parámetros y valor retornado.	
Parámetros por pila	El called ve dirección de retorno y accede parámetros/retorno con offsets como [RSP+8].	Registros usados solo como apoyo.
Parámetros por registros	La pila se usa menos; parámetros y retorno pueden ir en registros como CX, R8d, R9d.	CX lleva n; R8d y R9d retornan resultados.

getSumLimit: valor de retorno creado por el caller

Antes de llamar a getSumLimit, main hace `sub RSP, 8` para dejar espacio al retorno. Cuando entra al procedimiento, call ya puso la dirección de retorno en [RSP], por eso el espacio disponible para devolver el valor queda en [RSP+8].

```
getSumLimit PROC
    mov RBP, RSP
    ; RSP : Return Address
    ; RSP+8 : Returned Value

    mov dword PTR [RSP+8], limit
    ret
getSumLimit ENDP
```

Después, main hace pop RCX. Así recupera el valor que getSumLimit dejó en la pila y lo usa como parámetro n para los siguientes módulos.

sumCubes: ciclo con variables locales

sumCubes calcula iterativamente la suma: $1^3 + 2^3 + \dots + n^3$. En la versión limpia se usan dos variables locales de 32 bits: i y s.

```
sumCubes PROC
    mov RBP, RSP
    ; RBP : Direccion de retorno
    ; RBP+8 : Espacio para valor de retorno

    sub RSP, 8
    ; &i : RSP
    ; &s : RSP+4

    mov dword PTR [RSP], 0
    mov dword PTR [RSP+4], 0
```

```

cicloSumaCubos:
    xor EAX, EAX
    xor EDX, EDX
    inc dword PTR [RSP]    ; i = i + 1
    mov EBX, [RSP]
    mov EAX, EBX
    mul EBX                ; EAX = i^2
    mul EBX                ; EAX = i^3
    add [RSP+4], EAX       ; s = s + i^3
    cmp [RSP], ECX
    jne cicloSumaCubos

    xor R8, R8
    mov R8d, [RSP+4]

    mov RSP, RBP
    ret
sumCubos ENDP

```

- ECX/CX contiene n, el límite del ciclo.
- [RSP] funciona como i.
- [RSP+4] funciona como s, el acumulador.
- mul EBX usa implícitamente EAX como multiplicando y deja el resultado en EDX:EAX.
- Como el resultado útil se toma de EAX, se trabaja con valores pequeños en el ejemplo para evitar problemas prácticos.
- Al final se copia s a R8d como valor de retorno.

formulaCuadradoSuma: implementación útil para completar el ejercicio

En el PDF aparece como procedimiento pendiente. Para el trabajo, esta es una implementación directa de la fórmula usando n en CX y retornando el resultado en R9d:

```

formulaCuadradoSuma PROC
    mov RBP, RSP

    movzx EAX, CX    ; EAX = n
    mov EDX, EAX     ; EDX = n
    inc EDX          ; EDX = n + 1
    imul EAX, EDX     ; EAX = n * (n + 1)
    shr EAX, 1        ; EAX = n * (n + 1) / 2
    imul EAX, EAX     ; EAX = (n * (n + 1) / 2)^2

    xor R9, R9
    mov R9d, EAX     ; retorno en R9d

    mov RSP, RBP
    ret
formulaCuadradoSuma ENDP

```

Nota: shr EAX, 1 divide entre 2 desplazando a la derecha. Aquí funciona porque $n(n+1)$ siempre es par.

Depuración en Visual Studio: qué observar

- Colocar breakpoints en getSumLimit, sumCubes y antes del cmp final.
- Usar F11 para entrar a los procedimientos y ver cómo cambia RSP.
- En la ventana de registros, observar RAX, RBX, RCX, RDX, R8, R9, RSP y RBP.
- En la ventana de memoria, buscar la dirección de RSP para ver los valores locales i y s.
- Después de cada vuelta del ciclo, verificar que i aumente y que s acumule 1, 9, 36, 100, 225 para $n=1..5$.
- Antes del cmp final, R8 y R9 deberían tener el mismo valor si ambos cálculos están bien.

Valores esperados para probar rápido

n	Suma de cubos	Cuadrado de la suma	Resultado
1	1	1	Iguales
2	9	9	Iguales
3	36	36	Iguales
4	100	100	Iguales
5	225	225	Iguales
6	441	441	Iguales
7	784	784	Iguales
8	1296	1296	Iguales

Errores típicos que debes evitar

- Escribir kernel132.dll: lo correcto normalmente es kernel32.dll.
- Escribir CuadradoSuma dd3: debe ser CuadradoSuma dd 3, con espacio.
- Olvidar limpiar o definir el tamaño del dato al escribir memoria, por ejemplo dword PTR.
- No restaurar RSP antes de ret cuando el procedimiento reservó variables locales.
- Comparar registros de tamaños inconsistentes o dejar basura en la parte alta del registro.
- Confundir limit equ 5 con una variable en RAM: equ es una constante del ensamblador.
- Olvidar que call modifica la pila agregando la dirección de retorno.

Checklist de emergencia para el trabajo

- Identifica qué dato se pasa por registro y cuál por pila.
- Dibuja el stack frame antes de programar.
- Si reservas variables locales con `sub RSP`, elimina ese espacio con `mov RSP, RBP` antes de `ret`.
- Usa comentarios con los offsets: `&i = RSP`, `&s = RSP+4`, etc.
- Prueba con `n=5`: ambos métodos deben dar 225.
- Depura antes del `cmp R8,R9`: si uno falla, revisa primero el procedimiento que calculó ese registro.